

TemplateMethod

1. Explicación del patrón

Template Method es un patrón de comportamiento que define la estructura general de un algoritmo en una clase abstracta, pero permite que algunas etapas concretas de ese algoritmo sean implementadas por las subclases.

La idea principal es separar:

- La parte fija del algoritmo.
- Las partes variables que dependen de cada caso concreto.

La clase abstracta contiene un método principal, llamado método plantilla o método modelo, que marca el orden de los pasos del algoritmo.

Ese método llama a otros métodos más pequeños. Algunos de esos métodos pueden estar ya implementados en la clase abstracta, mientras que otros se dejan sin implementar para que las subclases los completen.

De esta forma, todas las subclases siguen la misma estructura general del algoritmo, pero pueden personalizar ciertos pasos.

2. Cuándo se utiliza

Se utiliza el patrón Template Method cuando:

- Varias clases tienen código idéntico que puede factorizarse en una clase común.
- Un algoritmo tiene una parte invariable y otras partes específicas según el tipo de objeto.
- Se quiere evitar duplicar código común en varias clases parecidas.
- Se quiere permitir que las subclases modifiquen solo ciertos pasos del algoritmo, pero no su estructura completa.
- Se quiere controlar desde una superclase el orden en el que se ejecutan los pasos.
- Existen varias versiones de un mismo proceso que comparten una estructura común.
- Se necesita que las subclases completen detalles concretos sin cambiar el flujo general.

3. Cómo aplicarlo

Para aplicar Template Method se suelen definir estos participantes:

Clase abstracta

Es la clase que contiene el método plantilla.

Responsabilidades:

- Definir el método principal del algoritmo.
- Establecer el orden de ejecución de los pasos.
- Declarar los métodos abstractos que deberán implementar las subclases.

- Implementar, si procede, los pasos comunes del algoritmo.
- Proporcionar métodos opcionales con implementación por defecto.

Ejemplo conceptual:

`ClaseAbstracta`

Contiene:

- `metodoPlantilla()`
- `pasoComun()`
- `pasoAbstracto1()`
- `pasoAbstracto2()`

El método plantilla no debería ser modificado por las subclases, porque representa la estructura fija del algoritmo.

Método plantilla

Es el método que define el esqueleto del algoritmo.

Responsabilidades:

- Llamar a los pasos del algoritmo en un orden concreto.
- Mantener fija la estructura general.
- Delegar en las subclases los pasos variables.

Ejemplo conceptual:

`metodoPlantilla()`

Puede hacer algo como:

1. Ejecutar un paso común.
2. Ejecutar un paso específico implementado por la subclase.
3. Ejecutar otro paso común.
4. Ejecutar otro paso específico.

Métodos abstractos

Son los pasos que la clase abstracta declara, pero no implementa.

Responsabilidad:

- Obligar a las subclases a proporcionar una implementación concreta.

Cada subclase concreta implementará estos pasos de forma distinta.

Métodos opcionales

Son pasos que tienen una implementación por defecto en la clase abstracta.

Las subclases pueden sobrescribirlos si necesitan cambiar ese comportamiento.

Ganchos o hooks

Son métodos opcionales que pueden estar vacíos por defecto.

Sirven como puntos de extensión para que las subclases puedan añadir comportamiento antes o después de pasos importantes del algoritmo.

El algoritmo puede funcionar aunque estos métodos no se sobrescriban.

Clase concreta

Es la subclase que completa las partes variables del algoritmo.

Responsabilidades:

- Implementar los métodos abstractos definidos por la clase abstracta.
- Sobrescribir métodos opcionales si necesita adaptar algún comportamiento.
- Respetar la estructura marcada por el método plantilla.

Puede haber varias clases concretas, cada una con una implementación diferente de los pasos variables.

4. Forma práctica de aplicarlo

Pasos:

1. Detectar varias clases con algoritmos parecidos o con código repetido.
2. Identificar qué partes del algoritmo son comunes.
3. Identificar qué partes cambian entre unas clases y otras.
4. Crear una clase abstracta común.
5. Mover el código común a esa clase abstracta.
6. Crear un método plantilla que defina el orden fijo del algoritmo.
7. Convertir los pasos variables en métodos abstractos.
8. Implementar esos métodos abstractos en cada subclase concreta.
9. Añadir métodos opcionales o hooks si se necesitan puntos de extensión.
10. Hacer que el cliente trabaje con la clase abstracta o con sus subclases sin duplicar el algoritmo completo.

5. Idea clave

La superclase controla la estructura del algoritmo.

Las subclases solo completan o modifican pasos concretos.

Así se reutiliza el código común y se evita que cada clase repita el mismo algoritmo con pequeñas variaciones.

6. Ejemplos del material

Ejemplo de pedidos

En el material aparece el ejemplo de una clase abstracta `Pedido` .

La clase abstracta contiene el método `modelo` y la firma de los métodos abstractos que ese método utiliza.

Después, las clases concretas como:

- `PedidoEspaña`
- `PedidoLuxemburgo`

implementan los métodos abstractos necesarios.

La estructura general del pedido se mantiene en la clase abstracta, mientras que los detalles concretos de cada país se implementan en las subclases.

Ejemplo de moneda

En el ejemplo del material aparece una clase general `Moneda` .

A partir de ella se derivan clases concretas como:

- `DolarAEuro`
- `DolarAFlorin`
- `EuroADolar`
- `EuroAFlorin`

La idea práctica es que existe una estructura común relacionada con la conversión de moneda, pero cada conversión concreta puede implementar sus pasos específicos.

7. Ventajas prácticas

- Evita duplicar código común en varias clases.
- Permite reutilizar la estructura general de un algoritmo.
- Facilita que las subclases personalicen solo los pasos necesarios.
- Mantiene el control del flujo principal en la clase abstracta.
- Permite añadir nuevas variantes creando nuevas subclases.
- Hace más fácil modificar la parte común del algoritmo en un único lugar.
- Ayuda a separar lo que es fijo de lo que es variable.

8. Cuidado

Template Method se basa en herencia.

Esto significa que las subclases quedan ligadas a la estructura definida por la clase abstracta.

Si el método plantilla tiene demasiados pasos, puede volverse difícil de mantener.

También hay que evitar que las subclases rompan la lógica esperada al sobrescribir pasos de una forma incompatible con el algoritmo general.

9. Diferencia práctica con Strategy

Template Method y Strategy permiten variar comportamientos, pero lo hacen de forma diferente.

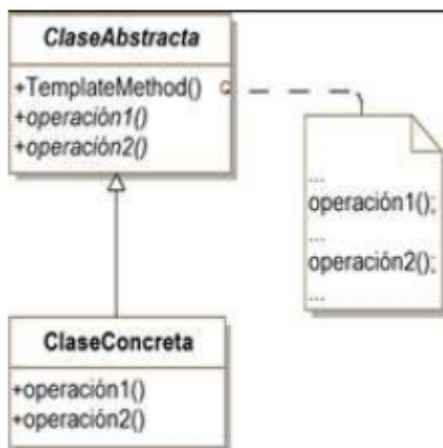
En Template Method, la variación se consigue mediante herencia. La clase abstracta define el algoritmo y las subclasses completan algunos pasos.

En Strategy, la variación se consigue mediante composición. El objeto principal recibe una estrategia distinta para cambiar su comportamiento.

Template Method trabaja más a nivel de clase.

Strategy trabaja más a nivel de objeto y permite cambiar el comportamiento en tiempo de ejecución con más facilidad.

10. Plantilla



11. Hueco para el ejemplo

